

芯鉴知微·技术架构说明

Oct 10, 2026 · @gusuisui

总体架构

系统是一个前后端分离的 Web 应用，加上一端的嵌入式设备；全部服务用 Docker Compose 一次启动。

组件	技术	端口/位置
数据库	PostgreSQL 16 (pgvector 镜像，仅为兼容历史迁移)	只在 Compose 内网
后端	Python + FastAPI, Uvicorn 运行，启动时自动执行数据库迁移	8000
前端	Vue 3 构建为静态文件，由 Nginx 提供并把 /api 代理到后端	8080
设备	ESP32 固件 (PlatformIO / Arduino)，HTTP(S) 上报	实验室局域网
模拟器	Python 命令行工具，生成带测试标记的设备数据	开发机

贯穿各层的设计原则只有一条：**证据先于结论**。设备上报、规则异常、故障候选和确认根因分层保存，任何结论都要引用本次实际入库的证据；证据不足时保留“未知”，不由 AI 或默认值补齐。它解释了后面很多取舍，比如为什么诊断用固定流程而不是自主 Agent。

后端

后端基于 FastAPI (0.142) 与 SQLAlchemy 2，用 Pydantic 2 做请求和配置校验，对外提供 /api/v1 下的 REST 接口。

层	位置	职责
路由	app/api/v1/routes/	认证、设备、学生、教师、诊断、知识、就绪等 13 组接口
业务服务	app/services/	设备接入、权限范围、实验会话、反馈、工单、记忆等业务规则

层	位置	职责
诊断与 AI	app/diagnosis/ 、 app/ai/	规则引擎、故障树、LangGraph workflow、模型调用治理
数据	app/models/ 、 migrations/	ORM 模型与 Alembic 迁移（当前到 0040）

几个工程约束由自动化门禁保证，不靠人工记忆：

- **接口登记**：每个接口都必须在 `access_policy.py` 登记鉴权方式，新增未登记的接口测试直接失败。
- **错误分类**：统一用 `core/errors.py` 区分临时故障、业务拒绝和程序缺陷；宽泛的异常捕获有白名单，只减不增。
- **权限复查**：写操作在事务内重新校验当前权限，避免等待期间权限变化导致越权。
- **幂等与回执**：设备上报、反馈、重新检查等命令都有请求身份，重发只返回原结果，客户端可以安全重试。

登录使用服务端会话令牌，失败次数在 PostgreSQL 里共享限流；设备用独立的设备 ID 和令牌认证，令牌只存 PBKDF2 哈希。

诊断与 AI

诊断是“确定性为主、AI 为辅”的固定流程：规则和故障树决定异常类型与候选范围，AI 只在这个范围内排序和写中文解释。流程由 **LangGraph**（1.2）编排，状态存在 PostgreSQL checkpoint 里，学生反馈后从原处恢复。

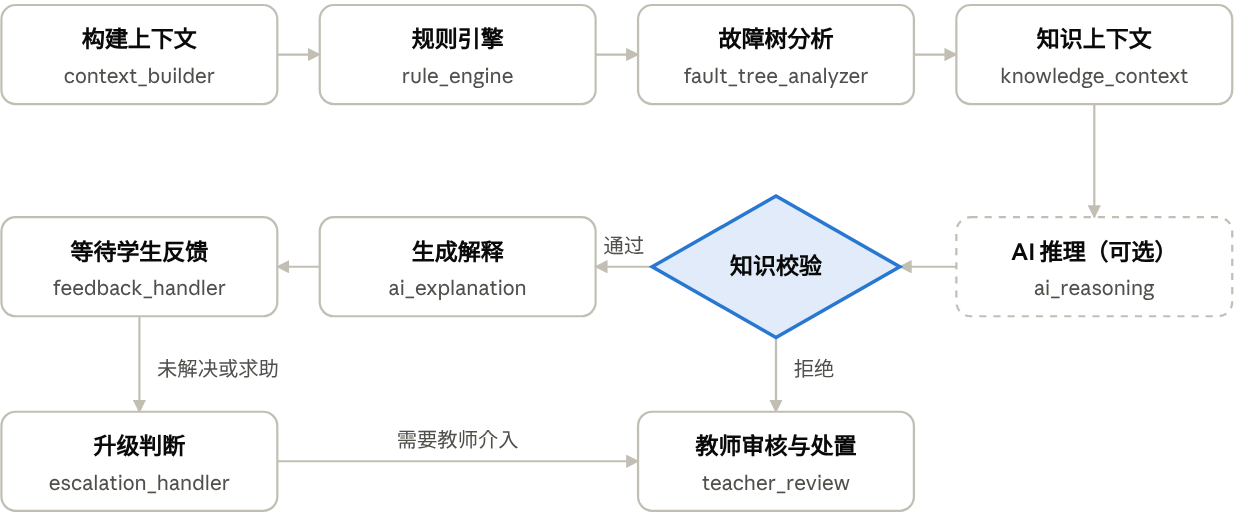
关键技术选择：

- **不用 RAG 和向量检索**：结构化案例规模有限，字段匹配就能说清匹配原因；故障类型和候选必须来自规则与故障树。保留的 `pgvector` 只为兼容历史迁移。
- **不用自主 Agent**：LangGraph 只负责固定节点和条件边，不让模型自己规划步骤或调用工具。
- **模型可替换**：通过 OpenAI 兼容接口接入，默认配置 DeepSeek，已用 Kimi（kimi-k2.6）做过真实合成案例实测。**AI 默认关闭**。
- **输出受合同约束**：模型按版本化 JSON Schema 输出，推理合同为 `evidence-reasoning-v2.14`；引用、候选和动作由独立的 `knowledge_validation` 节点核对，越界就拒绝。
- **调用治理**：外发前清洗敏感内容，按预算预留额度，每次调用记录模型、Prompt 版本、Token、耗时与降级原因；未配置、超时、限流、超预算或校验失败都自动退回确定性结果。

记忆分三类管理：**事实**（资料包里的固定配置）、**经验**（教师审核过的案例）、**工作**（当前任务的投影），来源停用后旧建议会被阻断，教师可以复核影响。

规则和故障树先定范围，AI 输出要过知识校验才到学生

LangGraph 固定节点；虚线框为可选的模型调用，关闭时走确定性结果



诊断工作流·固定节点与分支

知识校验不调用模型，只核对引用、候选和允许的动作；被拒绝的结果直接转教师，不会展示给学生。学生反馈“仍未解决”或求助后，由升级判断决定是否需要教师介入。

实验资料包与知识

每个实验的“领域知识”都装在一个版本化的**实验资料包**里，平台代码只负责执行，换实验不用改代码。

- **内容**：硬件配置、诊断规则、故障树、概念与案例、教学步骤、测试样例，共 10 个 YAML 文件；只允许数据，不允许脚本，Pydantic 按 `extra=forbid` 拒绝未知字段。
- **版本与快照**：服务端为每个文件和整包计算 SHA-256；学生开始实验时会话绑定当时发布的快照，资料包更新不会改写历史判断。
- **来源登记**：格式 1.1 为每条内容登记来源（如器件手册页码），案例带适用条件，调用模型前预检来源是否仍有效。
- **案例沉淀**：AI 可以起草案例，但必须经教师审核、事实确认后才进入“经验”；审批与来源在同一事务里完成。
- **受控查询**：DHT11 的“资料核对”由 `query_sources`（确定性来源服务）和 `query_tasks`（任务、题目、作答回执，迁移 0040）实现，不走自由检索。

当前有两个资料包：DHT11 温湿度 2.0.11 和 GPIO LED 2.0.4，都是待教师确认的测试草稿。

前端

前端是一个 Vue 3 单页应用，学生端和教师端共用一套工程和设计系统。

方面	技术
框架与语言	Vue 3.5 + TypeScript 5.8, Vite 7 构建
组件库	Element Plus 2.9
状态与路由	Pinia 3、Vue Router 4
图表	ECharts 6（设备状态、错误趋势、传感器曲线）
接口请求	Axios
测试	Vitest（组件与 store）、Playwright（浏览器端到端与真实后端集成）

页面分为四块：学生登录与工作台（当前实验 / 数据记录 / 实验参考）、教师登录与工作台（课堂处置 / 课堂概览 / 资料与审核）、以及公开的就绪门禁页。

设计上用统一的设计令牌（studio-system.css）：米白底、克莱因蓝主色、直角扁平风格，状态色分为正常绿、待核实橙、受阻红。工程上有三条约定：

- 后端的内部代码由 userLanguage.ts 统一投影成中文，不改变业务含义。
- 切换视图保留待确认的请求，过时的响应不覆盖新数据；401 重新登录、403 清理无权数据、409 核对状态。
- 测试、未知、缺证据、待确认等状态必须直接可见；采样时间和页面读取时间分开显示。

设备与固件

设备端是 ESP32-DevKitC V4 + DHT11，固件用 C++（Arduino on ESP-IDF）编写，PlatformIO 构建，当前版本 0.2.5。

- 采集**：按器件手册时序读取并校验，3 秒一次；读不到就报错，不编造读数。
- 上报**：设备协议 V1，一个接口 POST /api/v1/device/ingest 批量上报心跳、读数和日志；每批带 requestId，服务端幂等去重。

- **可靠性**：64 位单调时钟；待发批次先持久化到 LittleFS 再发送，确认后才清理；重试预算持久化，存储异常时停止采样而不自自动格式化。
- **安全**：HTTPS 必须配 CA 证书，明文 HTTP 只能在显式编译开关下用于隔离测试。

没有硬件时，**设备模拟器**（Python 命令行）提供 normal、read-failure、offline、recovery 等版本化场景，生成带测试标记的数据；固件源码另有主机模拟测试。硬件实操细节见《硬件开发指导》。

数据与安全

数据库是 PostgreSQL，当前 64 张表，由 40 个只追加的 Alembic 迁移管理。

数据域	代表表
身份与课堂	users、roles、auth_sessions、login_attempts、courses、classes、enrollments
设备与遥测	devices、device_bindings、device_heartbeats、device_logs、sensor_readings、ingestion_requests
实验与会话	experiments、experiment_versions、experiment_assignments、experiment_sessions
诊断与 workflow	diagnosis_results、diagnosis_evidence、diagnosis_issues、diagnosis_checks、diagnosis_workflow_runs、checkpoints
AI 与记忆	ai_call_records、ai_operations、ai_usage_reservations、knowledge_cases、memory_events
教师处置与审计	intervention_cases、classroom_messages、audit_events
受控查询	query_tasks、query_questions、query_answer_receipts

安全与数据保护：

- **备份恢复**：pg_dump 快照备份并生成内容清单（每表行数与全行 SHA-256），恢复演练在临时库里逐表核对。
- **凭据**：设备令牌与密码只存哈希；密钥只放在不提交的 .env；仓库有密钥扫描门禁。

- **权限与隐私**：学生按班级、任务、设备、会话授权，教师只看授权班级；私密备注按角色投影；发给模型的内容先清洗。
- **测试标记**：合成和演示数据带 `is_test_data`，前端明确显示，不混入正式统计。

质量保障

测试按层次分工，一键门禁 `scripts/verify.sh` 依次运行全部检查，任一步失败即停止。下表是 2026-10-10 的最近一次结果。

层次	工具	覆盖什么	最近结果
后端全量	pytest + 一次性 PostgreSQL	业务规则、鉴权、并发、迁移、 工作流	1782 项通过
前端单元	Vitest	组件与状态管理	184 项通过
浏览器端到端	Playwright（模拟接口）	页面行为与异常处理	22 项通过
真实后端集成	Playwright + 真实 FastAPI + PostgreSQL	浏览器 → 后端 → LangGraph → 数据库全链路	11 项通过
固件	主机模拟、协议样例校验	帧解码、时钟边界、缓存与回执 故障	通过（不替代 实物）

另有三道“棘轮”门禁防止质量回退：接口必须登记鉴权、测试数据必须由真实链路构造（手工构造白名单只减不增）、宽泛异常捕获白名单只减不增。执行与验收分离：验收方必须独立复跑，不凭执行方报告。

当前限制与演进方向

软件架构已经稳定，主要缺口都在“真实世界”的验证上。

限制	下一步
硬件尚未实物验收	按硬件指导完成 T1—T4 门槛，再做 LED 实验对照
资料包为测试草稿	教师确认接线、阈值与课程判据后发布正式版本
AI 效果只有合成案例证据	用教师独立保留集做语义验收，核实真实账单

限制	下一步
工作流 checkpoint 默认存内存	部署时切换到 PostgreSQL checkpoint 并验证重启恢复
未部署	确定服务器、HTTPS、监控告警、备份保留策略与负责人

有意不做事也写在这里：在出现独立需求和召回评测之前，不引入 RAG / 向量检索、自主规划 Agent 或学生画像。任何跨架构边界的改动，都要先形成可审查的决策，再同步合同、实现、测试和文档。